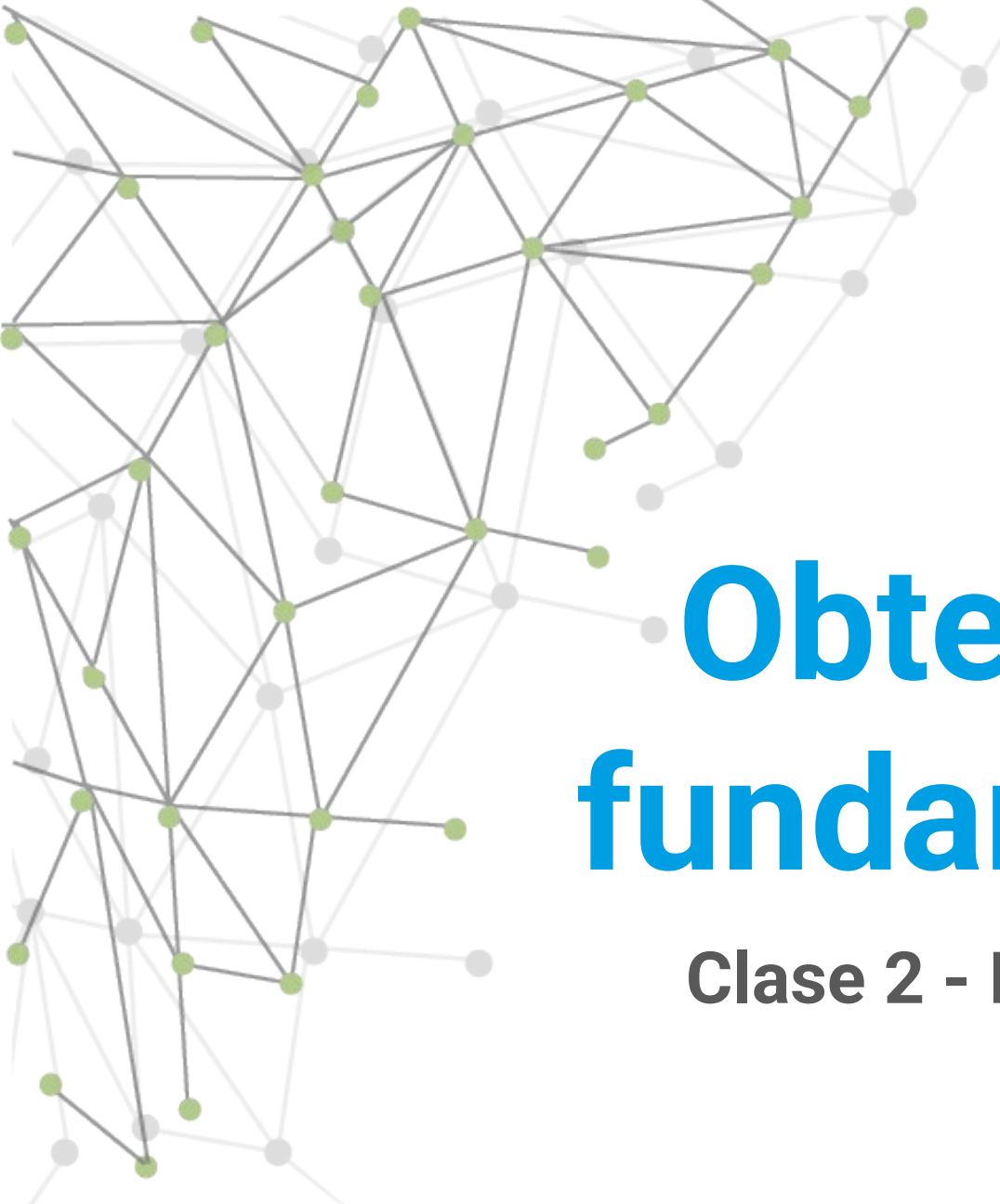
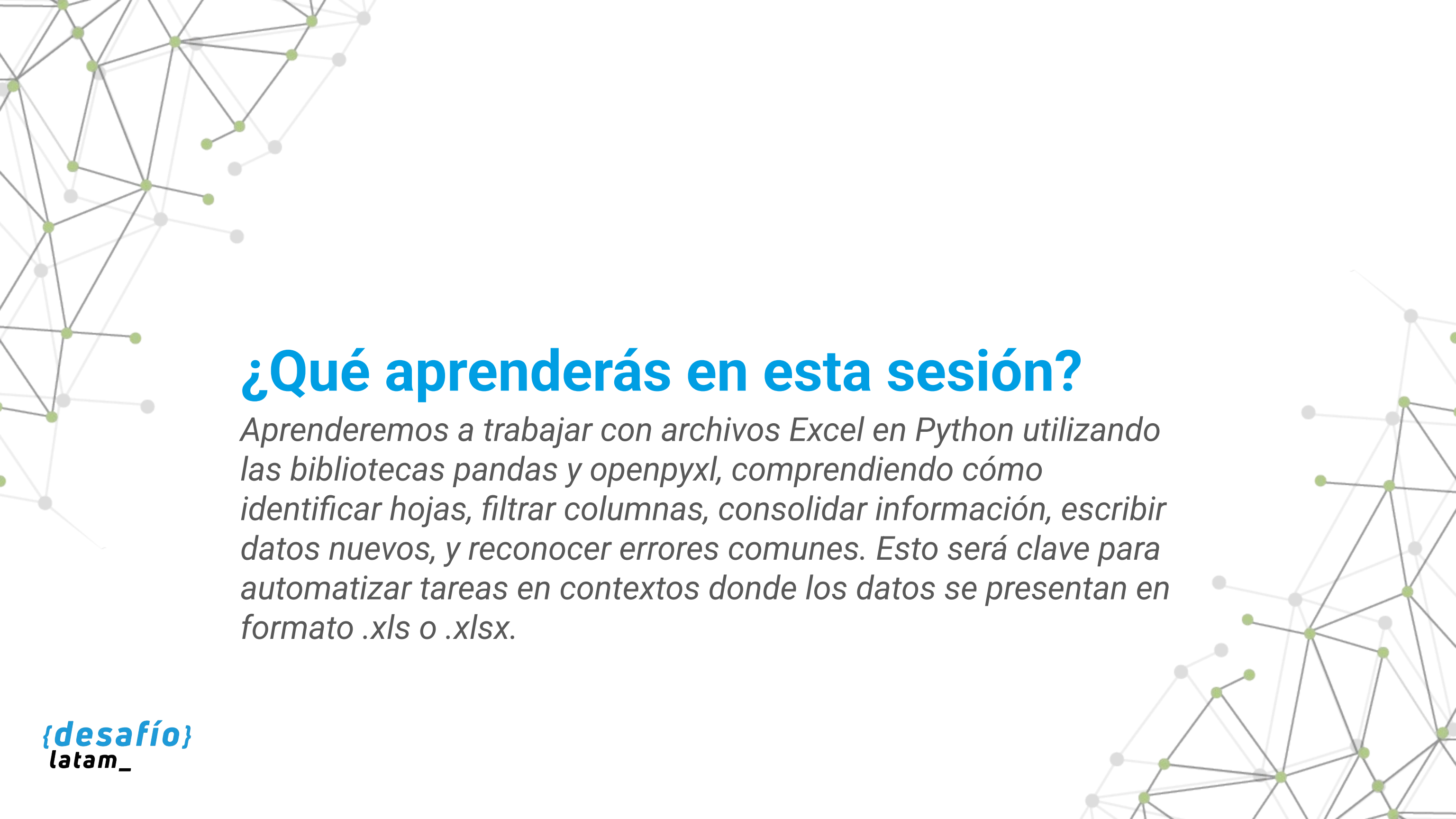




# Obtención de datos y fundamentos de Numpy

Clase 2 - Lectura y escritura de archivos Excel





## ¿Qué aprenderás en esta sesión?

*Aprenderemos a trabajar con archivos Excel en Python utilizando las bibliotecas pandas y openpyxl, comprendiendo cómo identificar hojas, filtrar columnas, consolidar información, escribir datos nuevos, y reconocer errores comunes. Esto será clave para automatizar tareas en contextos donde los datos se presentan en formato .xls o .xlsx.*

# Desafío inicial



## Problema

Eres parte del equipo de control de gestión de una empresa de servicios sanitarios. Cada mes, cada sucursal envía su reporte de consumo y reclamos en un archivo Excel que contiene varias hojas por comuna. Te han pedido consolidar esta información para generar un resumen automatizado de indicadores clave.



## Desafío

¿Cómo lo harías si tuvieras que repetir este trabajo cada mes, sin abrir uno a uno los archivos?



## Solución

Para resolver esta necesidad, deberás conocer cómo leer y escribir archivos Excel desde Python.

***/\* Archivos Excel: librería xlrd \*/***

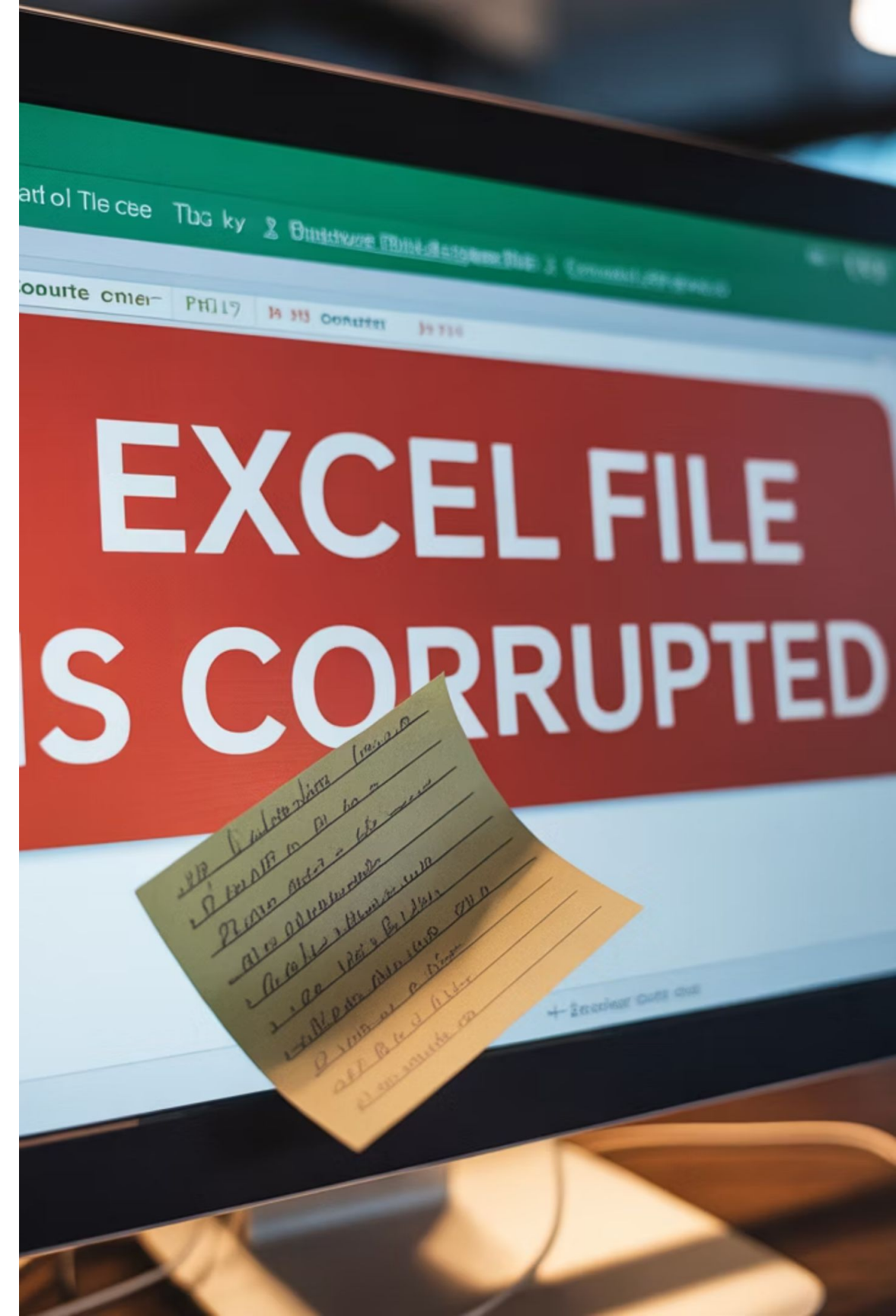


## ¿Qué es xlrd?

xlrd (Excel Read) es una biblioteca de Python que permite la lectura de archivos Excel antiguos en formato .xls (Excel 97-2003) y, hasta su versión 1.2.0, también era capaz de leer archivos en formato .xlsx. Fue ampliamente utilizada junto a pandas para importar hojas de cálculo directamente como DataFrames.

# ¿Qué pasó con xlrd y los archivos .xlsx?

- A partir de su versión 2.0.1 (diciembre de 2020), xlrd eliminó completamente el soporte para archivos .xlsx (basados en XML comprimido), debido a problemas de seguridad, mantenimiento del proyecto y dependencias complejas.
- Esto significa que intentar usar xlrd con archivos modernos generará un error, a menos que se utilice una versión antigua (como la 1.2.0).



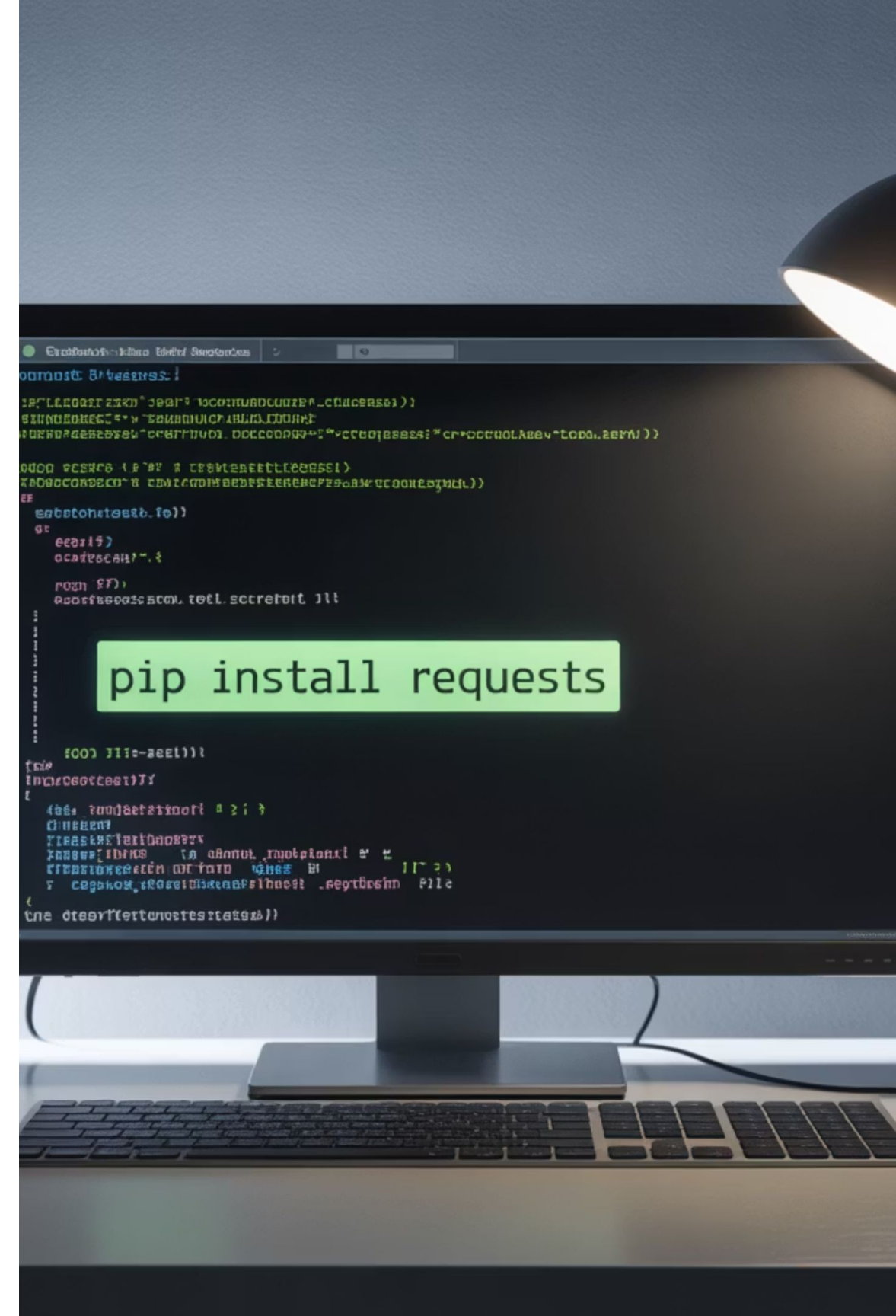


# Instalación de versión compatible

Ejemplo para instalar versión compatible con .xlsx:

```
pip install xlrd==1.2.0
```

Importante: No recomendado salvo en casos de compatibilidad con sistemas antiguos.



# ¿Qué alternativas se deben usar actualmente?

La forma moderna y segura de leer archivos Excel en formato .xlsx desde Python es usar pandas junto con el motor openpyxl. Esta biblioteca es activa, bien documentada y compatible con múltiples versiones modernas de Excel.

```
import pandas as pd
df = pd.read_excel('archivo.xlsx', engine='openpyxl')
```

Para exportar archivos Excel con formato visual (colores, estilos, etc.), se recomienda utilizar xlsxwriter como motor.



# Diferencias entre formatos .xls y .xlsx

| Característica     | .xls                       | .xlsx                              |
|--------------------|----------------------------|------------------------------------|
| Formato            | Binario propietario        | Basado en XML comprimido (OpenXML) |
| Compatibilidad     | Excel 97-2003              | Excel 2007 en adelante             |
| Tamaño de archivo  | Mayor                      | Menor, gracias a compresión        |
| Seguridad          | Baja (vulnerable a macros) | Mayor seguridad estructural        |
| Capacidad de datos | Limitada (65.536 filas)    | Ampliada (1.048.576 filas)         |

# ¿Cuándo se podría usar xlrd hoy?

- ☐ Cuando se trabaja con archivos legados de sistemas antiguos (.xls).
- ☐ Si se cuenta con bases históricas almacenadas en ese formato.
- ☐ En contextos cerrados donde la actualización tecnológica aún no es viable.



# Recomendación técnica actual

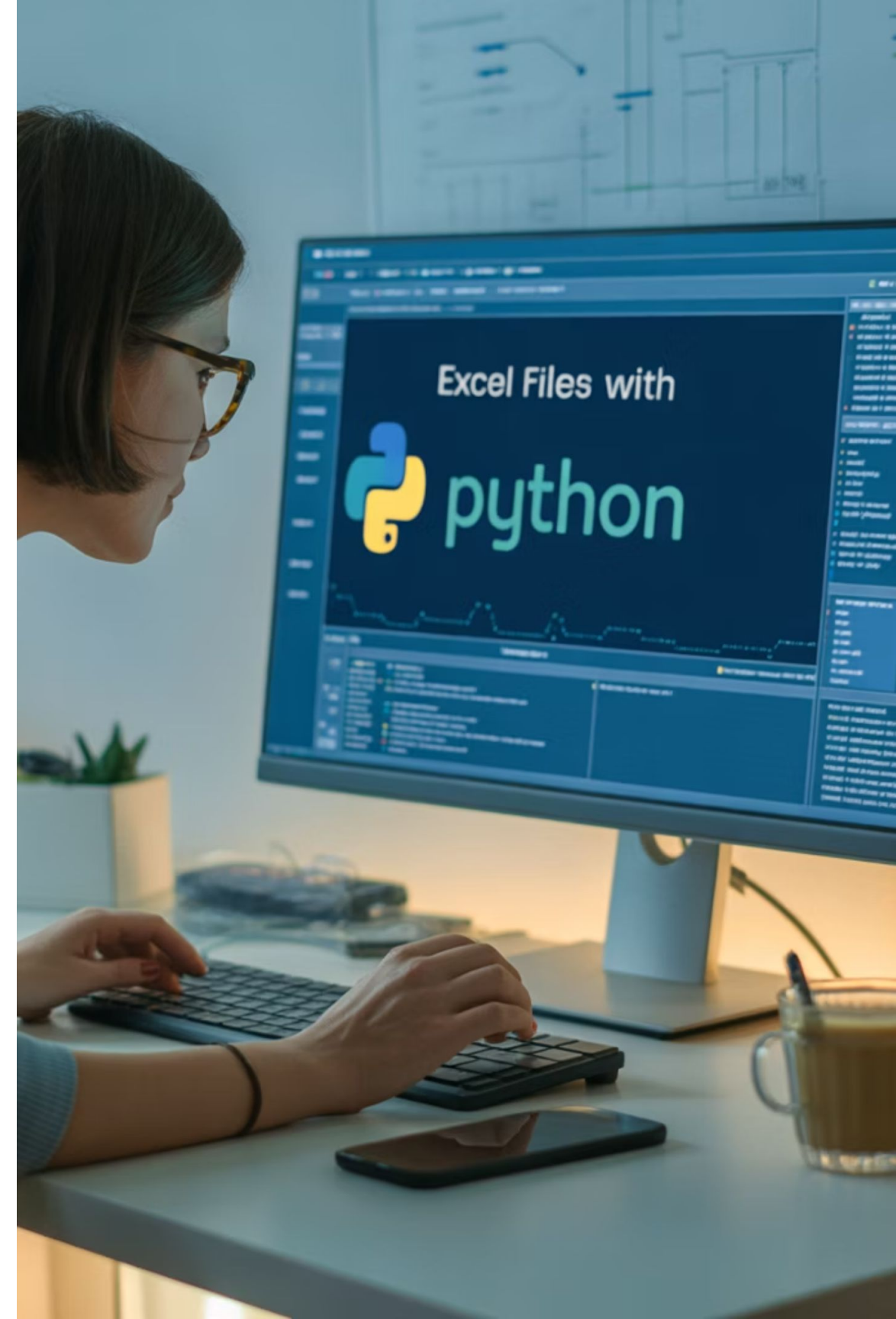
- Evitar xlrd para archivos .xlsx. No funcionará desde su versión 2.0.1 en adelante.
- Utilizar:
  - openpyxl para lectura y escritura moderna.
  - xlsxwriter cuando se requiere aplicar formatos visuales o preparar reportes presentables.
- Declarar el motor explícitamente en pandas para evitar advertencias o errores ambiguos.

***/\* Leyendo un archivo Excel \*/***

# Leyendo un archivo Excel

¿Por qué es importante leer archivos Excel en ciencia de datos?

El formato .xlsx sigue siendo uno de los más comunes para compartir reportes, indicadores y bases de datos intermedias. Aprender a leer eficientemente estos archivos en Python permite automatizar procesos repetitivos, consolidar datos desde múltiples fuentes y preparar información para su análisis posterior.



# Lectura básica de un archivo Excel

La forma más directa de leer un archivo Excel es mediante `pandas.read_excel()`. Se recomienda siempre declarar el motor explícitamente:

```
import pandas as pd
df = pd.read_excel('ventas.xlsx', engine='openpyxl')
print(df.head())
```

- `engine='openpyxl'` permite leer archivos `.xlsx` de forma estable y segura.
- `head()` muestra las primeras filas para inspeccionar rápidamente la estructura.



# ¿Cómo explorar las hojas disponibles?

Antes de leer datos, es importante conocer la estructura del archivo:

```
archivo = pd.ExcelFile('ventas.xlsx', engine='openpyxl')  
print(archivo.sheet_names)
```

Esto entrega una lista con los nombres de las hojas, útil cuando se desconoce el contenido o se necesita validar nombres antes de cargarlos.



# Leer una hoja específica

```
df_norte = pd.read_excel('ventas.xlsx', sheet_name='Región Norte', engine='openpyxl')
```

-  Carga solo la hoja necesaria para análisis.
-  sheet\_name puede ser el nombre exacto de la hoja o su posición (ej. sheet\_name=0).
-  Es útil para cargar solo los datos necesarios y optimizar la memoria.

# Leer columnas específicas

```
df = pd.read_excel('ventas.xlsx', usecols='A:C', engine='openpyxl')
```

Optimiza la carga de datos en memoria.

Permite centrarse en variables relevantes.

usecols puede recibir rangos ('A:C'), listas ([0, 2, 4]), o nombres (['producto', 'ventas']).

# Leer todas las hojas al mismo tiempo

Una función poderosa para tareas automatizadas:

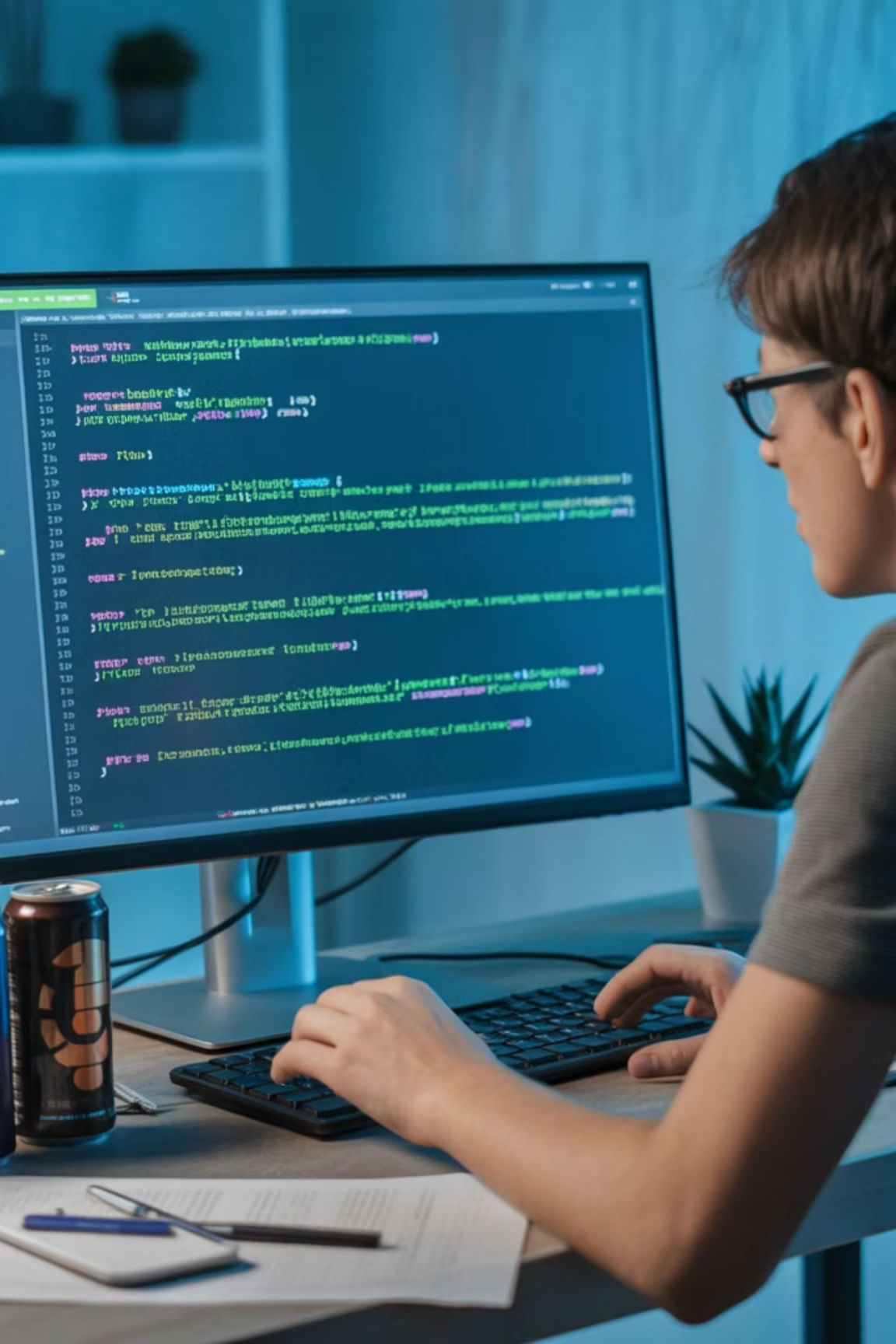
```
hojas = pd.read_excel('ventas.xlsx', sheet_name=None, engine='openpyxl')
for nombre, df in hojas.items():
    print(f"{nombre}: {df.shape}")
```

- sheet\_name=None carga todas las hojas como un diccionario (nombre\_hoja: DataFrame).
- Ideal para consolidaciones mensuales, reportes regionales o dashboards dinámicos.

# Errores frecuentes y soluciones

| Error                                               | Causa probable                              | Solución                                            |
|-----------------------------------------------------|---------------------------------------------|-----------------------------------------------------|
| ImportError: Missing optional dependency 'openpyxl' | pandas no encuentra el motor necesario      | Ejecutar pip install openpyxl                       |
| ValueError: Worksheet does not exist                | Nombre mal escrito o inexistente            | Verificar con .sheet_names antes de usar sheet_name |
| Header incorrecto                                   | La primera fila no contiene nombres válidos | Usar header=None o ajustar con skiprows             |
| ValueError: Usecols do not match columns            | Columnas mal referenciadas                  | Confirmar nombres con df.columns o cambiar usecols  |





# Buenas prácticas

- Verifica la estructura antes de cargar: usa `.sheet_names` y `.head()` para evitar errores.
- Maneja excepciones al leer múltiples hojas: `try...except` permite capturar errores sin detener el proceso.
- Valida los tipos de datos (`df.dtypes`) para evitar errores en análisis posteriores.



**/\* Escribiendo un archivo Excel \*/**

# Escribiendo un archivo Excel

¿Cómo puedes generar reportes Excel que otras personas de tu equipo puedan usar directamente?

Una de las tareas más comunes en análisis de datos es exportar resultados a un archivo Excel que pueda ser fácilmente interpretado y reutilizado por otras áreas de una organización (finanzas, operaciones, gerencia). Para esto, Python y pandas ofrecen un método eficiente y flexible.



# Exportar un DataFrame a Excel

```
df.to_excel('reporte.xlsx', index=False, engine='openpyxl')
```

- index=False evita que se agregue la columna de índices automáticamente.
- El parámetro engine='openpyxl' garantiza compatibilidad con .xlsx.
- Este archivo puede abrirse directamente en Excel por cualquier persona.

¿Qué pasaría si incluyes la columna de índice en un reporte que se entregará a gerencia?

Podría causar confusión o errores en la interpretación de filas si no corresponde a una variable significativa.

# Exportar columnas específicas

```
df[['producto', 'total']].to_excel('resumen.xlsx', index=False)
```

- Ideal para compartir solo los indicadores clave o para preparar reportes con foco en ciertas variables.
- También útil para construir hojas de resumen o visualizaciones interactivas en otros sistemas.



# Crear múltiples hojas en un mismo archivo

```
with pd.ExcelWriter('consolidado.xlsx', engine='openpyxl') as writer:  
    df1.to_excel(writer, sheet_name='Centro')  
    df2.to_excel(writer, sheet_name='Norte')
```

- ExcelWriter permite generar un único archivo con varias hojas.
- Muy útil para separar regiones, períodos, tipos de productos o fases del análisis.
- Puede integrarse en procesos automáticos de consolidación mensual.



# Validar el contenido exportado

Siempre es buena práctica verificar el archivo generado:

```
df_check = pd.read_excel('reporte.xlsx', engine='openpyxl')  
print(df_check.head())
```

- Esto permite confirmar que los datos fueron correctamente guardados.
- Es posible también usar `df.equals(df_check)` para comparar datos originales y exportados.





# Buenas prácticas al escribir archivos Excel

## Usar nombres descriptivos

Evita nombres genéricos como

datos\_finales.xlsx. Prefiere:

- reporte\_inventario\_mayo.xlsx
- ventas\_quincena1\_sucursalA.xlsx

## Incluir fechas o versiones

Esto facilita el orden cronológico y

evita sobrescritura accidental.

- proyeccion\_presupuesto\_2024-07-15.xlsx
- costos\_operativos\_v3.xlsx

## Evitar sobrescritura de archivos importantes

Previo a guardar, verifica si el archivo existe:

```
import os
if os.path.exists('resumen.xlsx'):
    print("¡Atención! Este archivo ya existe.")
```

# Buenas prácticas al escribir archivos Excel (continuación)

También puedes generar nombres dinámicos:

```
from datetime
import datetime
nombre =
f"resumen_{datetime.datetime.now().date()}.xlsx"df.to_excel(nombre,
index=False)
```

4. Evitar formato visual con pandas: `to_excel()` no aplica formatos (colores, bordes, negritas). Solo guarda datos.



# ¿Qué hacer si necesitas aplicar formato visual? (continuación)



Opción 2: xlsxwriter (crear desde cero con estilos)

```
with pd.ExcelWriter('formateado.xlsx', engine='xlsxwriter') as writer:  
    df.to_excel(writer, sheet_name='Resumen', index=False)  
    workbook = writer.book  
    worksheet = writer.sheets['Resumen']  
    formato = workbook.add_format({'bold': True, 'bg_color': '#F2F2F2'})  
    worksheet.set_row(0, None, formato)
```





# Formato visual en Excel - Consideraciones

Importante: Para aplicar estilos, es necesario trabajar directamente con los objetos workbook y worksheet del motor elegido, lo que implica conocimientos adicionales de esas bibliotecas.

- Requiere declarar el motor como 'xlsxwriter'.
- Permite aplicar negrita, color, bordes, alineación, entre otros estilos.
- Si necesitas un reporte presentable para ejecutivos, este enfoque es recomendable.



# Ejercicio guiado - Automatización de reportes regionales en Excel



# Ejercicio

## *Instrucciones*

**Objetivo:** Aplicar técnicas de lectura, exploración y escritura de archivos Excel utilizando pandas y openpyxl, simulando la consolidación de reportes de consumo eléctrico en una empresa con presencia nacional.

**Situación:** Trabajas en el área de planificación energética de una empresa que gestiona el consumo eléctrico residencial en distintas regiones del país. Cada mes, recibes un archivo en formato Excel con varias hojas, donde cada hoja representa los registros de consumo por ciudad.

Tu jefatura necesita generar un informe consolidado con el total de consumo por ciudad y por tipo de cliente (residencial o comercial), y espera que automatices el proceso para reutilizarlo cada mes.



# Ejercicio

## *Instrucciones*

Cargar el archivo y explorar sus hojas:

```
import pandas as pd
archivo = pd.ExcelFile('consumo_electrico_mensual.xlsx', engine='openpyxl')
print(archivo.sheet_names)
```

¿Cuántas hojas tiene este archivo? ¿Coincide con las ciudades que esperabas?

# Ejercicio

## Instrucciones

Leer todas las hojas y consolidar los datos:

```
hojas = pd.read_excel('consumo_electrico_mensual.xlsx', sheet_name=None,  
engine='openpyxl') # Agregar nombre de ciudad como columnadf_list = []  
for nombre_hoja, df in hojas.items():  
    df['ciudad'] = nombre_hoja  
    df_list.append(df)  
consolidado = pd.concat(df_list, ignore_index=True)  
print(consolidado.head())
```

# Ejercicio

## Instrucciones

Agrupar por ciudad y tipo de cliente:

```
resumen = consolidado.groupby(['ciudad',  
'tipo_cliente'])['consumo_kWh'].sum().reset_index()  
print(resumen)
```

¿Qué ciudad tiene el mayor consumo comercial? ¿Hay diferencia importante entre tipos de cliente?



# Ejercicio

## Instrucciones

Exportar a un nuevo archivo Excel:

```
resumen.to_excel('resumen_consumo.xlsx', index=False, engine='openpyxl')
```

Validar el archivo generado:

```
df_check = pd.read_excel('resumen_consumo.xlsx', engine='openpyxl')  
print(df_check)
```

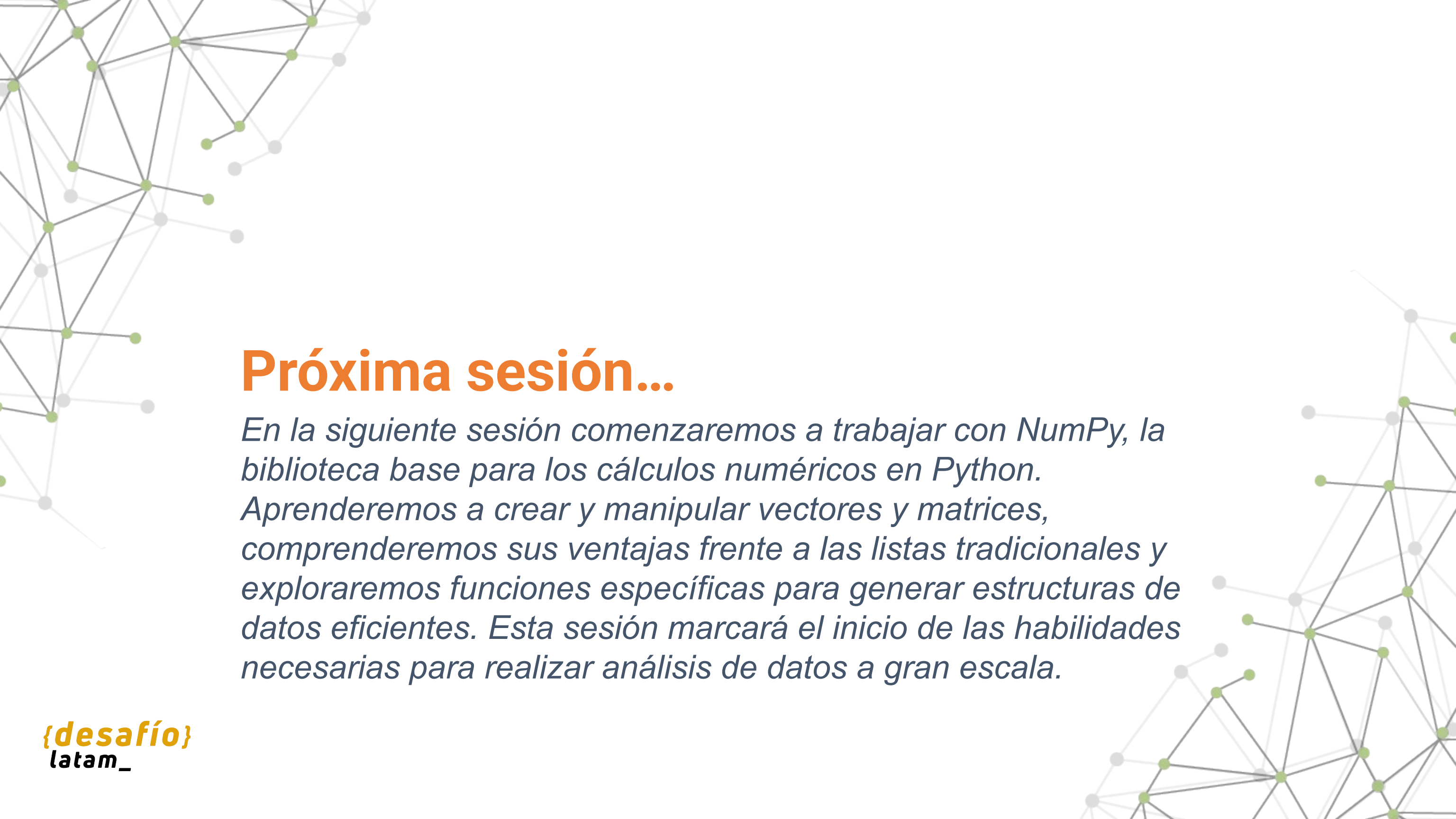
# Preguntas de reflexión y cierre

- ¿Qué ventajas tiene este enfoque respecto a copiar y pegar en Excel manualmente?
- ¿Cómo cambiarías el código si el archivo tuviera más de 10 hojas o nombres no estandarizados?
- ¿Qué otras variables podrían ser relevantes para la jefatura en este tipo de reportes?
- ¿Qué errores podrían ocurrir si no verificamos el contenido antes de escribir?



# Resumen

- 1** Aprendimos qué es un archivo Excel, sus formatos principales (.xls y .xlsx), y cómo se estructuran internamente.
- 2** Comprendimos por qué la biblioteca xlrd dejó de ser compatible con archivos .xlsx y por qué se recomienda utilizar openpyxl como motor de lectura y escritura.
- 3** Practicamos la lectura de archivos Excel con `pandas.read_excel()`, identificando hojas disponibles, seleccionando columnas y combinando múltiples hojas.
- 4** Exportamos DataFrames a archivos Excel utilizando `to_excel()` con buenas prácticas de nombrado, control de sobrescritura y estructura de múltiples hojas.
- 5** Identificamos errores frecuentes al trabajar con archivos Excel en Python y cómo prevenirlos o solucionarlos.



## Próxima sesión...

*En la siguiente sesión comenzaremos a trabajar con NumPy, la biblioteca base para los cálculos numéricos en Python. Aprenderemos a crear y manipular vectores y matrices, comprenderemos sus ventajas frente a las listas tradicionales y exploraremos funciones específicas para generar estructuras de datos eficientes. Esta sesión marcará el inicio de las habilidades necesarias para realizar análisis de datos a gran escala.*